Cletus: Section 1, Day 5

> To copy code: select it here, or click a "Copy on GitHub" link and use the copy button there. Full note online: https://github.com/mangucletus/AI-Engineering-Course/blob/main/section-1/cletus-section1-day5.md

- Date: [YYYY-MM-DD]
- Course: Week 1 Day 5, a company brochure generator using JSON prompts, chained LLM calls and streaming
- Lab notebook: week1/day5.ipynb and week1/week1 EXERCISE.ipynb (solution in week1/solutions/week1 SOLUTION.ipynb)
- Time spent: [e.g. 2h]

Key concepts

- The task: take a company name and website and write a brochure for customers, investors and job seekers.
- Chaining LLM calls: the first call picks the relevant links, and the second writes the brochure using those pages. One call's output becomes the next call's input, which is the first small step towards agentic AI.
- Zero-shot, one-shot and multi-shot prompting: putting 0, 1 or several examples of the output you want into the prompt. Day 5 uses one-shot, with one example of the JSON.
- JSON mode: response_format={"type": "json_object"} makes the model return valid JSON, which we read with json.loads(). A stricter version called Structured Outputs comes in Week 8.
- LLMs are good at judgement calls like deciding which links are "relevant". That would be really hard to code by hand.
- Truncating: cutting long text (for example [:5_000]) to keep tokens and cost down.
- Streaming: with stream=True the reply arrives in chunks while it's being written. Reading chunk.choices[0].delta.content gives the typewriter effect.
- Tone comes from the system prompt. Change it to "humorous and witty" and the same code makes a funny brochure.

Code / examples

The steps: get all the links on the page, let the LLM pick the relevant ones (as JSON), fetch those pages, have the LLM write the brochure, and stream it out.

The Python code runs the same on Linux and Windows. Run these in notebook cells inside the week1 folder, so the scraper import works.

Step 1, the LLM picks the links and answers in JSON:

Copy on GitHub

```
import json
from dotenv import load_dotenv
from openai import OpenAI
from scraper import fetch_website_links, fetch_website_contents

load_dotenv(override=True)
openai = OpenAI()

link_system_prompt = """You decide which links on a company website are relevant
for a brochure (About, Company, Careers pages). Respond in JSON like this example:
{"links": [{"type": "about page", "url": "https://full.url/about"}]}"""

def select_relevant_links(url):
    links = "\n".join(fetch_website_links(url))
    response = openai.chat.completions.create(
        model="gpt-5-nano",
        messages=[
            {"role": "system", "content": link_system_prompt},
            {"role": "user", "content": f"Links on {url} (some relative):\n{links}"},
        ],
        response_format={"type": "json_object"},
    )
    return json.loads(response.choices[0].message.content)

print(select_relevant_links("https://huggingface.co"))
```

Step 2 (run after step 1) puts the landing page and each relevant page into one user prompt, then streams the brochure back:

Copy on GitHub

```
brochure_system_prompt = """You write a short brochure about a company for
customers, investors and recruits. Use the website contents you are given.
Respond in markdown."""

def stream_brochure(company_name, url):
    pages = "Landing page:\n" + fetch_website_contents(url)
    for link in select_relevant_links(url)["links"]:
        pages += f"\n\n{link['type']}:\n" + fetch_website_contents(link["url"])
    messages = [
        {"role": "system", "content": brochure_system_prompt},
        {"role": "user", "content": f"Company: {company_name}\n\n{pages[:5_000]}"},
    ]
    stream = openai.chat.completions.create(
        model="gpt-4.1-mini", messages=messages, stream=True
    )
    for chunk in stream:
        print(chunk.choices[0].delta.content or "", end="")

stream_brochure("HuggingFace", "https://huggingface.co")
```

Exercise / challenge

Week 1 challenge, a technical tutor. It takes a technical question and answers it with GPT (streamed) and with Llama 3.2 (local), so we can compare the two. Ollama needs to be running for the Llama part (the check commands are in my Day 2 notes).

Copy on GitHub

```
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv(override=True)
openai = OpenAI()
ollama = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")

question = """Please explain what this code does and why:
yield from {book.get("author") for book in books if book.get("author")}"""

system_prompt = (
    "You are a helpful technical tutor for Python, "
    "software engineering, data science and LLMs."
)
messages = [
    {"role": "system", "content": system_prompt},
    {"role": "user", "content": question},
]

print("GPT:\n")
stream = openai.chat.completions.create(
    model="gpt-4o-mini", messages=messages, stream=True
)
for chunk in stream:
    print(chunk.choices[0].delta.content or "", end="")

print("\n\nLlama 3.2:\n")
response = ollama.chat.completions.create(model="llama3.2", messages=messages)
print(response.choices[0].message.content)
```

If Llama 3.2 isn't downloaded yet (same on Linux and Windows):

Copy on GitHub

```
ollama pull llama3.2
```

More practice: - Make a brochure for a different company. - Add a third call that translates the brochure into French. - Replace the fixed question with input() so the tutor asks you for one.

Questions & blockers

Things to watch out for: - JSON mode needs the word "JSON" somewhere in the messages, otherwise OpenAI returns an error. - Some links fail to scrape. Relative links like /about have to become full URLs, and the prompt asks the LLM to do that. - Long pages use a lot of tokens, so truncate the input.

To discuss with my partner

- Other places content generation could help at work: marketing copy, personalised emails, tutorials written from a spec.
- What should go in our Week 1 summary.md?

Resources

- The scraping helpers: week1/scraper.py (fetch_website_links and fetch_website_contents)
- OpenAI prompting guide: https://platform.openai.com/docs/guides/prompt-engineering
- Other students' projects: week1/community-contributions/